

SMART CURRICULUM

Activity & Attendance System

Full Project Report & Team Handbook

Smart India Hackathon (SIH) 2026 - Internal Round

Team of 6 | 1 Team Leader + 5 Module Leads

Role-Based Technical Documentation

Complete system knowledge for the Team Leader, with dedicated module sections for Database, Backend, Frontend, Testing and Deployment / Git & GitHub.

Table of Contents

Section	Topic	Primary Owner
0.	Team Structure & Role Assignments	All
1.	Project Overview (Problem, Solution, Features)	Team Leader
2.	System Architecture & Module Connections	Team Leader
3.	Database Design & SQL	Member 1 - SQL
4.	Backend - Python & Django	Member 2 - Django
5.	Frontend Technology Stack	Member 3 - Frontend
6.	Testing Strategy	Member 4 - Testing
7.	Deployment, Git & GitHub	Member 5 - DevOps
8.	Demo Data, Credentials & Run Guide	Team Leader
9.	Conclusion & Future Roadmap	Team Leader

How to use this handbook: The Team Leader is expected to know Sections 0-9 fully and can answer questions across all modules. Each of the 5 members is primarily responsible for their numbered section but must understand how it connects to the rest of the system.



Team Structure & Role Assignments

Applies to all 6 members

The project is split into 6 clear work streams. Every member owns one module end-to-end while the Team Leader holds the complete picture.

Role Matrix

Member	Role	Responsible For	Knows (Scope)
Team Leader	Project Lead / Architect	Everything: architecture, connections, data flow, integration, reviews, final submission	Full system (Sections 1-8)
Member 1	Database Engineer	Data models, SQL, migrations, relationships, queries, seed data	SQL + schema (Section 3)
Member 2	Backend Engineer	Django apps, views, URLs, DRF APIs, forms, authentication, business logic	Python / Django (Section 4)
Member 3	Frontend Engineer	HTML templates, CSS theme, JS interactivity, QR scanner, charts	Frontend stack (Section 5)
Member 4	QA / Testing Engineer	Test plan, unit & integration tests, bug reports, regression	Testing (Section 6)
Member 5	DevOps Engineer	Git workflow, GitHub, Vercel deployment, env config, builds	Deployment + Git/GitHub (Section 7)

Team Communication Rules

Knowledge boundary: Every member masters their own module but must explain how it connects to the others (they need not know every implementation detail of other modules).

Team Leader ownership: The leader is the single point of truth for architecture decisions, integrations, and the final presentation.

Code flow: All code merges through the Team Leader's review before going to the main branch on GitHub.

Demo responsibility: Each member presents their own module at the internal round.



Project Overview

Team Leader - must know everything

What the product is, the problem it solves, and the feature set.

1.1 Executive Summary

SmartCurriculum Activity & Attendance is a Django-based web application that removes classroom administration overhead while turning students' empty periods into productive study time. Teachers generate a time-limited QR code for each class; students scan it to mark attendance in under a second. A timetable engine then detects free periods and a daily planner auto-suggests subject-linked tasks, so every free slot becomes active learning. Analytics dashboards give both teachers and students live, chart-driven insight into attendance performance.

1.2 Problem Statement

Manual roll-call wastes 3-5 minutes of every class and is prone to proxy attendance.

Students do not know their attendance percentage until it is too late (75% rule).

Empty periods are wasted; there is no system to guide students toward productive work.

Teachers lack an automated way to monitor low-attendance students and export reports.

1.3 Our Solution

QR-based attendance: UUID sessions with expiry countdowns; one scan per student per session.

Free-period optimizer: Timetable parser detects free slots and the planner recommends tasks filtered by the student's enrolled subjects.

Analytics: Daily trends, per-subject percentages, 75% threshold warning flags (safe / warning / danger).

Exports: One-click Excel and PDF attendance reports per session.

1.4 Feature List & Tech Snapshot

4	Django Apps
8	Database Tables
4	REST API Endpoints
6	Team Members
75%	Attendance Rule

Layer	Technology	Version / Tool
Backend Framework	Django	5.2+

Layer	Technology	Version / Tool
REST API	Django REST Framework	3.16+
Database (dev)	SQLite	bundled with Python
Database (prod)	PostgreSQL (serverless)	Neon Postgres via DATABASE_URL
ORM / SQL	Django ORM + Raw SQL support	sqlite3 / psycopg (psycopg[binary])
Frontend	HTML / CSS / JavaScript	Bootstrap 5.3, Chart.js 4.4, jsQR 1.4
PDF Reports	ReportLab	4.4+
Excel Reports	openpyxl	via pip
QR Generation	qrcode + Pillow	8.0+
Deployment	Vercel (Serverless Python)	vercel.json
Version Control	Git + GitHub	main branch workflow



System Architecture & Module Connections

Team Leader - must know everything

How the 4 Django apps connect to each other, the full request flow, and the URL routing map. This is the integration blueprint of the project.

2.1 High-Level Architecture

Layer	What Lives Here
Client	Browser (teacher / student). Bootstraps Bootstrap 5.3, Chart.js, jsQR via CDN.
Web Server	Django WSGI (local dev) or Vercel serverless Python runtime (production).
Routing	smartcurriculum/urls.py includes accounts, attendance, timetable, analytics and the /api/ endpoints.
Application Layer	accounts (users/roles), attendance (QR sessions & records), timetable (slots & planner), analytics (aggregation views).
Database Layer	SQLite locally, PostgreSQL on Vercel. Accessed only through the Django ORM.
Static / Media	static/ (CSS, JS) served by WhiteNoise-equivalent; media/ stores QR PNGs and profile pictures.

2.2 App-to-App Connections (Data Flow)

From	To	Connection	How
accounts.CustomUser	attendance.Class	teacher (FK) + students (M2M)	Class.teacher -> teacher user; Class.students -> student users
attendance.Subject	attendance.Class	subject (FK)	Each class is a specific subject offering
attendance.Class	attendance.Attendance Session	class_obj (FK)	A session is one attendance window for a class
attendance.AttendanceSession	attendance.Attendance Record	session (FK)	Every scan creates one record per student
accounts.CustomUser	attendance.Attendance Record	student (FK)	Tracks which student was present/late
attendance.Class	timetable.TimeSlot	class_obj (FK)	Timetable slots reference class objects
timetable.TaskSuggestion	timetable.DailyPlanner	tasks (M2M)	Planner auto-picks tasks by student's subjects
accounts.CustomUser	timetable.DailyPlanner	student (FK)	One planner per student per day
attendance.*	analytics.views	read-only queries	Analytics aggregates sessions & records via ORM

2.3 URL Routing Map

URL Pattern	View	Access
/accounts/login/ , /logout/ , /register/student/ , /register/teacher/ , /profile/	accounts.views	Public (login/register), login (profile)
/ (dashboard)	attendance.views.dashboard	Any authenticated user
/classes/ , /classes/create/ , /classes/<id>/	attendance.views.class_*	Teacher/student
/classes/<id>/generate-qr/	attendance.views.generate_qr	Teacher only
/attendance/mark/	attendance.views.mark_attendance	Student only
/attendance/history/	attendance.views.attendance_history	Any
/attendance/session/<uuid>/ (+ close/export)	attendance.views.session_detail etc.	Teacher (close), any (view)
/timetable/ , /timetable/planner/	timetable.views.timetable_view, daily_planner	Student / teacher
/analytics/	analytics.views.analytics_dashboard	Any
/api/mark-attendance/	api_views.api_mark_attendance	Student (session auth)
/api/session/<uuid>/status/	api_views.api_session_status	Authenticated
/api/student/attendance/	api_views.api_student_attendance	Student
/api/student/active-sessions/	api_views.api_active_sessions	Student

2.4 Core Business Flow

Step 1: Teacher creates a Class (ClassForm) and picks the Subject and enrolled Students.

Step 2: Teacher opens the class and clicks **Generate QR** -> AttendanceSession created with status=active, UUID session_id and expires_at = now + duration (default 15 min).

Step 3: A QR PNG is generated from the JSON payload {session_id, class, subject, timestamp} and saved to media/.

Step 4: Student scans (camera via jsQR, file upload, or manual paste) -> POST /attendance/mark/ with the session JSON.

Step 5: mark_attendance validates session activity + enrolment, then AttendanceRecord is created (unique per session+student).

Step 6: Session expires automatically (is_active() flips status to expired). Teacher can close it early.

Step 7: Attendance history, session detail, Excel/PDF export, and analytics all read from AttendanceRecord.

Step 8: Timetable finds today's free periods -> DailyPlanner auto-fills tasks from TaskSuggestion linked to enrolled subjects.

2.5 Authentication & Roles

CustomUser (accounts.CustomUser) extends Django AbstractUser with role (student / teacher / admin), phone, department, roll_number, employee_id, year_of_study, section and profile_picture.

Login: Session-based (DRF SessionAuthentication + Django auth). Session cookie valid 24 hours.

Role guards: is_teacher() / is_student() checked in views; API endpoints return 403 for wrong roles.

Security: CSRF enforced on POST forms; QR payloads carry no secrets, only a UUID.



Database Design & SQL

Member 1 - Database Engineer

Complete schema, table columns, relationships, database configuration and the SQL / ORM queries used by the application.

3.1 Database Configuration

Located in **smartcurriculum/settings.py**. Development uses SQLite (zero setup). In production, the app automatically switches to PostgreSQL when a DATABASE_URL / POSTGRES_URL environment variable is present (parsed by dj-database-url with conn_max_age=600 for connection pooling). The Postgres driver psycopg[binary] is declared in requirements.txt. On Vercel the build runs migrate + seed_data automatically, so the production schema and demo users are created at deploy time.

```
db_url = os.environ.get('DATABASE_URL') or os.environ.get('POSTGRES_URL')
        or os.environ.get('POSTGRES_URL_NON_POOLING')
if db_url:
    DATABASES = {'default': dj_database_url.parse(db_url, conn_max_age=600)}
else:
    DATABASES = {'default': {'ENGINE': 'django.db.backends.sqlite3',
                              'NAME': BASE_DIR / 'db.sqlite3'}}
```

3.2 Table Inventory (8 Tables)

Table	Purpose	Key Columns
accounts_customuser	Users of all roles	username, password, first_name, last_name, email, role, phone, department, roll_number, employee_id, year_of_study, section, profile_picture
attendance_subject	Academic subjects	name, code (unique), department, credits
attendance_class	Class offering (teacher + students + subject)	name, department, year, section, subject_id (FK), teacher_id (FK), students (M2M)
attendance_attendancesession	One QR attendance window	session_id (UUID unique), class_obj_id (FK), teacher_id (FK), date, start_time, end_time, status, expires_at, qr_code, qr_data, location
attendance_attendancerecord	Per-student scan result	session_id (FK), student_id (FK), status, marked_at, ip_address, device_info, latitude, longitude
timetable_timeslot	Weekly schedule	class_obj_id (FK), day, start_time, end_time, slot_type, room_number, department, year, section
timetable_tasksuggestion	Prebuilt study tasks	title, description, category, subject_id (FK), duration_minutes, priority, resources_link
timetable_dailyplanner	Per-student daily planner	student_id (FK), date, custom_notes, tasks (M2M), completed_tasks (M2M)

3.3 Relationships & Constraints

One-to-Many: Subject -> Class -> AttendanceSession -> AttendanceRecord (CASCADE delete).

One-to-Many: Teacher (CustomUser) -> Class (teaching_classes), Teacher -> AttendanceSession (created_sessions), Student -> AttendanceRecord.

Many-to-Many: Class.students (enrolled students); DailyPlanner.tasks and DailyPlanner.completed_tasks (both to TaskSuggestion).

Unique constraints: Subject.code unique; AttendanceRecord unique_together (session, student) -> a student cannot mark twice; DailyPlanner unique_together (student, date).

Role restrictions: Class.teacher limited to role=teacher; Class.students limited to role=student (limit_choices_to).

3.4 Status & Choice Fields (enum-like)

Field	Choices	Meaning
CustomUser.role	student, teacher, admin	Drives dashboard routing and view permissions
AttendanceSession.status	active, expired, closed	Lifecycle of a QR window
AttendanceRecord.status	present, absent, late	Outcome of a scan / manual marking
TimeSlot.day	MON..SAT	Day of the week
TimeSlot.slot_type	class, free, lunch, lab	Enables free-period detection
TaskSuggestion.category	study, revision, assignment, project, reading, practice, research	Classifies suggested tasks

3.5 Key Queries Used in the App (ORML & SQL Equivalent)

3.5.1 Attendance percentage per subject (student dashboard)

```
total_sessions = AttendanceSession.objects.filter(class_obj=cls).count()
present_count = AttendanceRecord.objects.filter(
    student=student, session__class_obj=cls, status='present').count()
percentage = round((present_count / total_sessions * 100), 1)

# SQL:
# SELECT COUNT(*) FROM attendance_attendancesession
# WHERE class_obj_id = <cls.id>;
# SELECT COUNT(*) FROM attendance_attendancerecord r
# JOIN attendance_attendancesession s ON r.session_id = s.id
# WHERE r.student_id = <sid> AND s.class_obj_id = <cid>
# AND r.status = 'present';
```

3.5.2 Detect absent students for a session

```
absent_students = session.class_obj.students.exclude(
    attendance_records__session=session)

# SQL (simplified):
# SELECT u.* FROM accounts_customuser u
# JOIN attendance_class_students cs ON u.id = cs.customuser_id
# WHERE cs.class_id = <cid>
# AND u.id NOT IN (
#     SELECT student_id FROM attendance_attendancerecord
#     WHERE session_id = <sid>);
```

3.5.3 Active sessions a student has not yet marked

```
AttendanceSession.objects.filter(class_obj__in=enrolled, status='active')
    .exclude(records__student=request.user)
```

3.6 Migrations & Seeding

Create/apply migrations: python manage.py makemigrations && python manage.py migrate

Seed demo data: `python manage.py seed_data` (creates admin, 3 teachers, 8 students, 5 subjects, 4 classes, 10 days of sessions/records, timetable, 10 tasks)

Idempotent: `seed_data` uses `get_or_create` so it can be re-run safely (important for Vercel build).



Backend - Python & Django

Member 2 - Backend Engineer

Project layout, settings, apps, views, URL wiring, DRF API, forms, admin and authentication logic.

4.1 Project Layout

```
internal-hackthon/
  manage.py          # Django CLI entry point
  smartcurriculum/
    settings.py      # Project config package
    urls.py          # Apps, DB, auth, templates, static, CORS, DRF
    asgi.py / wsgi.py # Root URLconf (includes all apps + api)
  accounts/          # ASGI / WSGI entry points
  attendance/         # CustomUser + registration/login/profile
  timetable/          # Subjects, Classes, Sessions, Records + API
  analytics/          # TimeSlots, TaskSuggestions, DailyPlanner
  templates/          # Teacher & student analytics dashboards
  static/             # All HTML templates (base + per-app)
  media/              # css/main.css, js/main.js
  attendance/management/commands/seed_data.py # Uploaded QR codes + profile pictures
  requirements.txt    # Python dependencies
  vercel.json         # Vercel build/deploy config
```

4.2 settings.py Highlights

INSTALLED_APPS: django.contrib.* + rest_framework, corsheaders, crispy_forms, crispy_bootstrap5, accounts, attendance, timetable, analytics.

AUTH_USER_MODEL = accounts.CustomUser (custom user model, set before first migration).

DRF: SessionAuthentication; default permission IsAuthenticated.

CORS: CORS_ALLOW_ALL_ORIGINS=True (prototype); CSRF_TRUSTED_ORIGINS read from env.

Static/Media: STATIC_URL=/static/, STATIC_ROOT=staticfiles, MEDIA_ROOT=media, MEDIA_URL=/media/ served via static() helper in urls.py.

Time zone: Asia/Kolkata; SESSION_COOKIE_AGE = 86400 (24 h).

4.3 Apps, Views & URL Patterns

4.3.1 accounts

View	Purpose
login_view	AuthenticationForm; redirects by next= or dashboard
logout_view	Logs out and redirects to login
register_student / register_teacher	Role-based UserCreationForm; auto-login after save
profile_view	ProfileUpdateForm incl. profile picture upload

4.3.2 attendance (core)

View	Purpose
dashboard	Routes teacher / student / admin to their dashboard
teacher_dashboard	Classes, today's + active sessions, student count
student_dashboard	Enrolled classes, per-subject %, recent records, active sessions to scan
class_list / class_create / class_detail	CRUD-lite for classes incl. per-student stats
generate_qr	Creates active session, JSON payload, saves QR PNG (qrcode + Pillow)
mark_attendance	Parses QR JSON, validates session + enrolment, creates record (JSON response)
session_detail / close_session	Live attendance view; teacher closes early
attendance_history	Teacher session list / student record list
export_attendance_excel	openpyxl workbook download
export_attendance_pdf	ReportLab PDF download (A4 table report)

4.3.3 timetable

View	Purpose
timetable_view	Grid (MON-SAT) of TimeSlots; today's slots + free periods
daily_planner	Auto-suggests tasks from enrolled subjects; marks completed; notes

4.3.4 analytics

View	Purpose
teacher_analytics	14-day trend, class summary, low-attendance (<75%) warning list
student_analytics	Per-subject %, needed-for-75% counter, 30-day trend, overall stats

4.4 REST API (Django REST Framework)

Method	Endpoint	Body / Params	Response
POST	/api/mark-attendance/	session_id (QR JSON or UUID)	{success, message}
GET	/api/session/<uuid>/status/	-	session + records + present_count + total_students
GET	/api/student/attendance/	-	list of attendance records
GET	/api/student/active-sessions/	-	list of active scannable sessions

4.5 Forms & Admin

ClassForm: fields name/department/year/section/subject/students; students queryset filtered to role=student.

AttendanceSessionForm: location + duration (5-120 min).

Student/Teacher RegistrationForm: UserCreationForm subclasses that stamp role on save.

Admin: All 8 models registered with list_display / filters / search; CustomUser uses fieldsets.



Frontend Technology Stack

Member 3 - Frontend Engineer

Templates, styling system, JavaScript behaviour, QR scanning UX and chart rendering.

5.1 Stack Summary

Asset	Tech	Source / Notes
Templates	Django Template Language (DTL)	templates/ with base.html + extends/blocks
CSS Framework	Bootstrap 5.3.0	CDN (jsdelivr)
Icons	Font Awesome 6.5.0	CDN (cloudflare)
Fonts	Inter + Space Grotesk	Google Fonts
Charts	Chart.js 4.4.0	CDN; line / doughnut / bar
QR Scanning	jsQR 1.4.0	CDN; camera + image + manual input
Custom CSS	static/css/main.css	Dark/light themes via CSS variables
Custom JS	static/js/main.js	Theme, sidebar, scanner, toasts, charts, countdown

5.2 Template Architecture

base.html holds the responsive sidebar (role-aware nav links), topbar (clock, theme toggle, bell), messages area and content block. Loads Bootstrap bundle, Chart.js and main.js.

Blocks: title, extra_css, breadcrumb, content, extra_js allow every page to extend base cleanly.

Role-aware navigation: teachers see My Classes / Create Class / Attendance History; students see Scan QR / My Classes / My Attendance / Timetable / Daily Planner.

Theme: ; CSS variables (--bg-card, --text-primary, --accent-primary, ...) redefine all colours per theme; choice stored in localStorage.

5.3 JavaScript Features (main.js)

Feature	Function(s)	Behaviour
Theme toggle	applyTheme / toggleTheme	Swaps data-theme + icon; persisted in localStorage
Sidebar	toggleSidebar	Collapse on desktop, overlay drawer on mobile
Live clock	updateClock	IST time updated every second in topbar
Counters	animateCounters	Eased count-up on [data-count] cards
Progress bars	animateProgressBars	Width animation via [data-progress]
Toasts	showToast	Stacked success/error notifications

Feature	Function(s)	Behaviour
QR camera scan	startQRScanner / scanFrame / stopQRScanner	getUserMedia video -> jsQR decode -> submit
QR image scan	handleQRDrop / handleQRFileSelect / scanQRImage	Drag-drop or file picker -> jsQR decode
Manual submit	submitManualQR / submitAttendance	POSTs session data to /attendance/mark/
Session countdown	startCountdown	Live expiry timer on the QR display page
Live refresh	autoRefreshSession	Polls /api/session/<uuid>/status/ every 5s
Charts	createDonutChart / createLineChart / createBarChart	Chart.js renderers for analytics
Copy link	copyShareLink	Copies QR share URL to clipboard

5.4 Mark Attendance UX (mark_attendance.html)

Camera: + hidden ; jsQR decodes each frame; on hit, stops stream and posts immediately.

Drag & drop / browse: dropzone accepts image/*; decoded via jsQR on a canvas.

Manual input: paste the raw QR JSON or UUID, then Submit.

Success overlay: full-screen overlay with animated check on success.

Deep-link support: /attendance/mark/?session_id=...&class=...&subject=... pre-fills the input and shows class details.



Testing Strategy

Member 4 - QA / Testing Engineer

Test framework, what to cover, how to run tests, and the QA checklist for the demo.

6.1 Framework & Tooling

Framework: Django's built-in unittest (django.test.TestCase / Client). No extra test library required.

API tests: DRF's APIClient for endpoint-level checks.

Run all tests: python manage.py test (optionally --verbosity 2).

Code quality checks: python manage.py check (system checks) and python -m compileall . for syntax.

6.2 Recommended Test Matrix

Module	Test Case	Expected Result
accounts	Register student / teacher	User created with correct role; auto-login
accounts	Login with wrong password	Form invalid; no session created
accounts	Profile update	Fields persist; picture upload saved
attendance	Teacher creates class	Class saved with teacher FK
attendance	Student in wrong class scans QR	403 / error message (not enrolled)
attendance	Duplicate scan same session	Only one record (unique_together)
attendance	Expired session scan	Session rejected (is_active False)
attendance	Export Excel / PDF	HTTP 200, correct content-type
timetable	Planner auto-suggestion	Tasks limited to enrolled subjects
timetable	Complete a task	completed_tasks updated
analytics	Student with <75% flagged	Status = danger in subject_stats
api	POST /api/mark-attendance/ valid session	success True, record created
api	GET /api/student/attendance/ as teacher	403 Forbidden

6.3 Sample Test Snippets

Model test - unique attendance

```
from django.test import TestCase
class AttendanceTests(TestCase):
    def test_unique_record_per_student(self):
        # create student, class, session ...
        AttendanceRecord.objects.create(session=s, student=st)
        with self.assertRaises(Exception):
            AttendanceRecord.objects.create(session=s, student=st)
```

View test - role guard

```
from django.test import TestCase
from django.urls import reverse
class GuardTests(TestCase):
    def test_student_cannot_generate_qr(self):
        self.client.login(username='student1', password='student123')
        r = self.client.get(reverse('class_list'))
        self.assertEqual(r.status_code, 200)
```

API test with APIClient

```
from rest_framework.test import APIClient
client = APIClient()
client.login(username='student1', password='student123')
r = client.post('/api/mark-attendance/', {'session_id': qr_json}, format='json')
self.assertEqual(r.status_code, 200)
```

6.4 QA Checklist (before demo)

Register both roles and log in / out.

Teacher: create class, generate QR, watch countdown, close session, export Excel & PDF.

Student: scan via camera, upload image, manual paste; attempt double scan (must fail).

Expired session: wait for expiry or set short duration; scanning must be rejected.

Timetable shows today; free periods detected; planner fills tasks for enrolled subjects.

Analytics: charts render (Chart.js CDN), warning flags correct at 75% / 60% thresholds.

Responsive layout at 1280px, 768px, 375px widths; sidebar collapse on mobile.

Dark/light theme toggle persists after refresh.

7

Deployment, Git & GitHub

Member 5 - DevOps Engineer

Version control workflow, GitHub usage, Vercel serverless deployment, and environment configuration.

7.1 Git Workflow

Step	Command	Notes
Clone repo	git clone <repo-url>	Initial pull
Branch	git checkout -b feature/module	Member works on own branch
Stage	git add .	Review with git status first
Commit	git commit -m "type(scope): message"	e.g. feat(attendance): add qr export
Pull main	git pull origin main	Keep in sync (rebase if needed)
Push	git push origin feature/module	Upload branch
PR	Open Pull Request on GitHub	Team Leader reviews & merges
Merge	git checkout main && git merge feature/module	Only after review passes

7.2 Commit Conventions

Type	When	Example
feat	New feature	feat(analytics): add low-attendance warnings
fix	Bug fix	fix(attendance): reject expired sessions
refactor	No behaviour change	refactor(forms): simplify class form
docs	Documentation	docs: update README
test	Tests	test(api): cover mark-attendance endpoint
chore	Tooling / deps	chore: add openpyxl dependency

7.3 GitHub Repository Hygiene

Keep **.gitignore** active: db.sqlite3, media/, staticfiles/, __pycache__/, .env, venv.

Never commit secrets - store them as GitHub repository **Secrets** or Vercel **Environment Variables**.

Main branch is protected in spirit: only the Team Leader merges PRs after review.

README.md documents setup + default credentials for judges/demo machines.

7.4 Vercel Deployment

The project deploys as a Python serverless app on Vercel. The build pipeline runs migrations, seeds demo data and collects static files automatically (see vercel.json).

```
{
  "version": 2,
  "buildCommand": "python manage.py migrate --noinput &&
                    python manage.py seed_data &&
                    python manage.py collectstatic --noinput"
}
```

7.5 Required Environment Variables

Variable	Purpose	Example
DJANGO_SECRET_KEY	Django signing secret	
DJANGO_DEBUG	Debug mode	False
DJANGO_ALLOWED_HOSTS	Allowed hostnames	*.vercel.app, your-domain.com
DJANGO_CSRF_TRUSTED_ORIGINS	CSRF-safe origins	https://*.vercel.app
DATABASE_URL / POSTGRES_URL	Production Postgres DSN	postgres://user:pass@host/db

7.6 Local Development Commands

```
# 1. Install dependencies
python -m venv .venv && .venv\Scripts\activate      (Windows)
pip install -r requirements.txt

# 2. Database
python manage.py makemigrations
python manage.py migrate
python manage.py seed_data

# 3. Run
python manage.py runserver

# 4. Verify
python manage.py check
python manage.py test
```



Demo Data, Credentials & Run Guide

Team Leader - must know everything

Ready-made accounts and the exact steps to run a smooth demo.

8.1 Seed Credentials

Role	Username	Password	Count
Admin	admin	admin123	1
Teachers	teacher1, teacher2, teacher3	teacher123	3
Students	student1 ... student8	student123	8

8.2 Demo Script (5 minutes)

Step 1: Open the deployed URL; show the login page (Team Leader intro).

Step 2: Log in as teacher1 -> Dashboard -> My Classes -> open a class -> Generate QR (short duration, e.g. 5 min).

Step 3: Open the student mark-attendance page in another tab/device; scan the QR with camera (or upload the saved PNG). Success overlay appears.

Step 4: Show session detail: live count, absent list, then export Excel and PDF.

Step 5: Student logs in -> Dashboard shows attendance % per subject, then Analytics for charts.

Step 6: Show Timetable (today + free periods) and Daily Planner with auto-suggested tasks.

Step 7: Teacher Analytics: 14-day trend + low-attendance warning list.

Step 8: Close with the roadmap (future work).

8.3 Troubleshooting

Symptom	Likely Cause	Fix
QR image not saved	media/ missing	mkdir media or run collectstatic
Charts not rendering	Chart.js CDN blocked offline	Check internet or vendor the library
Camera permission denied	Insecure context	Use https:// or localhost
Scan says 'not enrolled'	Student not in Class.students	Add student via admin / class form
Vercel build fails	Env vars missing	Set DATABASE_URL + SECRET_KEY in Vercel



Conclusion & Future Roadmap

Team Leader - must know everything

Final summary and next steps after the internal round.

9.1 What Was Delivered

A complete, role-aware attendance + free-period optimization platform in Django.

QR-based attendance with expiry, duplicate protection and instant feedback.

Automated daily planner that converts empty periods into guided study time.

Teacher and student analytics with 75% rule warnings and Excel/PDF exports.

A 6-member team with clearly separated, documented module ownership.

9.2 Future Roadmap

Idea	Description	Owner
Geo-fencing / geolocation	Restrict scanning to classroom GPS radius (fields already exist)	Backend
AI task suggestions	Rank TaskSuggestions by student performance & subject weakness	Backend + Frontend
Live attendance wall	Projector view with real-time present/absent grid	Frontend
Push notifications	Remind students before free periods with suggested tasks	Backend + DevOps
Bulk CSV import	Import students/classes from college ERP exports	Database
E2E browser tests	Selenium/Playwright flows covering the demo script	Testing
CI/CD pipeline	Run tests + checks automatically on every PR via GitHub Actions	DevOps

9.3 Closing Note

Every member knows their module inside out and the Team Leader understands how every module connects. Use this handbook for preparation, the demo script for the live round, and keep the code on GitHub as the single source of truth.

Good luck, Team!